

Cyberattack event logs classification using deep learning with semantic feature analysis

Ahmad Alzu'bi^{a,*}, Omar Darwish^b, Amjad Albashayreh^a, Yahya Tashtoush^a

^a Department of Computer Science, Jordan University of Science and Technology, Irbid, Jordan

^b Information Security and Applied Computing, Eastern Michigan University, Ypsilanti, USA

ARTICLE INFO

Keywords:

Cyberattack
Event logs
Intrusion detection
Deep learning
BERT

ABSTRACT

Event logs play a crucial role in cybersecurity by detecting potentially malicious network activities and preventing data loss or theft. Previous work did not place a high value on log messages and their impact on security breach prediction and intrusion detection. This research paper introduces a novel approach for log message analysis applied to a dataset of event logs collected from various web sources. Event log messages were analyzed and categorized based on event and attack types with an explainable AI emphasizing the value of its key data. The study aims to enhance intrusion detection and minimize performance degradation by identifying suspicious events. In this regard, a new semantic vectorization framework is proposed, leveraging deep learning architectures to develop semantic discriminating log features, offering a cogent explanation and classification of event log messages. The use of BERT deep embeddings as a baseline for the prediction model allows for visualizing and interpreting the formulation of log message semantic features. Several empirical scenarios are set and conducted extensively to evaluate the performance of the event log classifier, considering the attack type, event type, and zero-shot logs. The experimental results demonstrate that the proposed event log classifier outperforms state-of-the-art machine learning models, achieving a recall of 99.27% and a precision of 99.29%. This highlights the model's ability to accurately identify events of a particular type by detecting as many suspicious events as feasible while minimizing the misclassification rate.

1. Introduction

The recent growth of the Internet and the rapid expansion of technologies make a considerable increase in cyberattacks, exposing individuals and companies to increased security threats. The complexity of the threat landscape has intensified concomitantly with the escalating frequency of cyberattacks, thereby resulting in an increasing number of serious threats to critical infrastructure (Zhang et al., 2020). Much of the useful information related to security threats, vulnerabilities, and attacks can be found in the event log files. Therefore, to tackle the growing cyber threats, there is a demand to collect the current logs as soon as feasible and assess the potential impact of the network and service events that are recorded in the log files. Security event logging and monitoring are two components of the same process that are crucial for having a secure infrastructure.

Log files record all the events that occur in a system based on the log severity level. To accomplish this, the implemented logging facilities gather, collect, and save logs generated continuously by most devices and parts of the system (Azahari and Balzarotti, 2024; Satpathi et al., 2019). This includes getting permission to access, delete, add an

application or file, change the system date, shut down the system, and modify the configuration settings. Event logs record both user actions and those performed by processes running on the computer (Satpathi et al., 2019). The event log is also an important source for evaluating the system's safety.

Event logs provide the administrators with critical data to solve any problem with the system (Ali et al., 2022). This data could be a textual description of the event, when it occurred, and who was responsible for it. Moreover, logs enable administrators to validate process performance and identify bottlenecks. Aside from these functional benefits, since log files include a large number of recurring lines, they can be efficiently compressed and recorded affordably (Landauer et al., 2020b). Categorizing events based on their types could be helpful with issues like monitoring critical events, knowing what software and services are installed on the system, boot events, processing events, types of crashes, and clearing events that occurred. The primary benefit of the categorization process lies in its ability to assist in the identification and classification of attacks. An attacker typically enters the system with previously stolen credentials from a valid user and thus behaves

* Corresponding author.

E-mail address: agalzubi@just.edu.jo (A. Alzu'bi).

similarly to a malicious insider 97% of the time (Jaafar et al., 2016). With the extensive use of computer networks, cyberattacks are becoming increasingly common as attackers exploit system vulnerabilities for sensitive data theft, monetary gains, or even complete network infrastructure destruction (Singh et al., 2017). As a result, there is an urgent need to protect the targeted systems from penetration, which can be done by monitoring and understanding the systems' events.

The event log message provides a detailed description of the specific activity that took place. Textual log messages are in a human-readable, unencoded format, and they encompass information that can be utilized to ascertain the type of attack that occurred. Consequently, these messages serve as a means to identify, categorize, and timestamp attacks. Even though log traces are increasingly being used to detect and prevent unauthorized access to corporate information systems (Padiet et al., 2024), there are limited datasets available for anomaly-based intrusion detection techniques (Ring et al., 2017). Moreover, the previous research has not solely relied on semantic information in log messages, a valuable aspect of the attack detection process. For instance, the researchers in (Landauer et al., 2022; Lin et al., 2022) obtained multiple network flow features such as timestamps, byte count, source/destination information, and messages containing specific parameters as the primary method for identifying abnormal behavior in the network. Most importantly, previous studies employed a conventional classification method to assess the normality or suspicion of event logs and network flows (Satilmiş et al., 2024). The data originated from a testbed established by the same individuals who recorded and categorized the events. This led to an expected manner for managing the attacks and predicting their type based on the employed hosts and the length of time that the attack was implemented. This implies that the attacks were captured based on time-specific scenarios under controlled settings, and the process was fully supervised (Tovarňák et al., 2020). Furthermore, certain event log datasets (Tovarňák et al., 2020; MACCDC, 2012) have been introduced recently by capturing event logs, yet without classifying them into specific attack types. In contrast, our study uses a variety of data sources from different operating systems and network traffic types to categorize events based on attack and event types. Unlike traditional approaches, our methodology prioritizes semantic features over non-semantic ones, making the classification process more challenging by assessing the model's ability to identify event or attack types solely from the message text.

Earlier investigations (Kumar et al., 2023; Kalutharage et al., 2023; Das et al., 2021) employed explainable AI (XAI) techniques to elucidate the contribution of numeric features in network flows for a specific type of attack. However, these studies did not delve into the semantic meaning of the textual content within event logs. The XAI tools aid in identifying key features in vectorized log message representations, providing insights into the most influential semantic elements for attack detection. The interpretability of the vectorization process enhances the comprehension of how specific text features contribute to categorizing various types of attacks.

As a result, this study suggests a novel approach for higher-level data classification, emphasizing the importance of the semantic content of event messages in detecting intrusion. This is achieved by utilizing a diverse dataset of log events collected from various source environments, which undergoes systematic processing and analysis intending to detect attacks. The proposed approach provides detailed information for classification procedures, aiding in attack-type detection. Different explainable AI techniques are employed to assess the impact of log words in the message text for predicting the attack type. Subsequently, a vectorization framework is established to evaluate the classifier's capacity to learn a specific schema for decision-making, enabling it to scrutinize the semantic content of the message for predicting attack types. The main contribution of this work is three-fold:

1. Introducing a novel classification approach for event logs based on the semantics embedded in the log messages. This approach is validated on a new collection of event logs categorized by event and attack types.
2. Proposing a semantic vectorization framework based on deep learning methods as a foundation for developing semantic vectors to provide solid analysis of event log messages, as well as utilizing BERT capabilities in visualizing the attribution of the message's semantic features.
3. Providing a profound XAI-based analysis of the labeled log messages with machine learning algorithms, including a hybrid deep architecture, CNN with GRU.

The rest of this paper is organized as follows: Section 2 reviews the related work; Section 3 presents a new labeled dataset of event log messages and the procedure of log labeling and processing. Section 4 discusses the methodology of the proposed work; Section 5 describes the experimental results with a thorough discussion, and Section 6 concludes this paper.

2. Related work

In recent years, cybersecurity researchers and practitioners have developed a wide range of methods (Ramish et al., 2022; Landauer et al., 2020a) that use machine learning to detect and classify intrusions at the network and host levels in a timely and autonomous manner. Popular research initiatives in cyberattack detection strategies include: the insider threat analysis technique based on kernel PCA and LSTM-RNN that was developed by Meng et al. (2018). One of the newest malicious traffic detection systems is OADSD (Zhang et al., 2023). OADSD works based on three steps: collecting the network traffic, extracting the important features, and detecting the anomalies. CTSCOPY (Mei et al., 2021) is an example as a provenance graph-based technique for cyber threat detection that uses system-level event data streams to train an anomaly detection model. There was an primary emphasize on constructing and upholding datasets suitable for training and evaluating cybersecurity models and algorithms. Datasets such as ToN IoT dataset (Moustafa et al., 2020), game Theory and cybersecurity (GTCS) (Mahfouz et al., 2020), and ADFA linux dataset (ADFA-LD12) (Crech and Hu, 2013) are critical for developing effective cyber defense strategies and assessing the effectiveness of existing security measures. Table 1 summarizes the research works related to the scope of this study by grouping them based on their primary contributions into three categories: event log classification, cyberattack detection techniques, and the creation of new cyberattack datasets.

To the best of our knowledge, there is no existing research that has devised a novel cyberattack detection technique with a newly diverse dataset of event logs, leveraging the semantic features of event messages. Furthermore, there is no comprehensive dataset that includes a wide range of attack types. The majority of the existing data classified the event as normal or suspicious, and a few datasets classified the event based on the attack type with limited labels. The research studies did not examine how much log messages contribute to the prediction process. Their technical focus was primarily on classifying the logs into specific attack types. However, no effort was made to understand the predictive nature of the attack types or the rationale behind classifying them into specific types as opposed to others. Their attention was centered on the outcome without delving into the underlying reasons. Conventional event log classification did not consider the individual contribution of log messages in generating each label, or how each word in the entire text of the message contributes to the prediction process specifically.

In contrast to earlier studies, this paper proposes a new cyberattack detection framework and creates a dataset of log events from several source environments. This study also distinguishes itself by incorporating various approaches from previous work, such as categorization schema and semantic log vectorization, and incorporating

Table 1

A summary of the recent related works.

Aim	Ref	Methods	Type of attacks	Limitations
Logs Classification	Ramish et al. (2022)	KNN, RF, SVM, MLP	Brute Force, SQL Injection	Purely based on binary classification
	Gunawan et al. (2018)	Kali Linux VM	SQL Injection, XSS, WPA2	The victim side only contains Windows event logs.
	Ceron et al. (2020)	MikroTik honeypot	Brute Force, Botnet	Attacker avoidance of cloud-based honeypots and non-router devices
	Vu et al. (2019)	LR, NN, LightGBM, XGBoost	–	Purely based on binary classification
Cyberattack Detection	Zhang et al. (2023)	OADS, SVM, NB	DDoS, DoS	Deep learning not utilized
	Sriram et al. (2020)	LSTM, RF, DT, NB, KNN, SVM, LR	DoS, Reconnaissance, DDoS	Purely based on network flow data.
	Vinayakumar et al. (2019)	DNN	DoS, Probe, R2L, U2R	Reliance on predefined features and labeled data
	Mei et al. (2021)	CTSCOPY	Ransomware, RCE, Phishing, Data Theft	Limited number of event types
	Das et al. (2020)	LR, NB, DT, NN, SVM	SQL Injection, CRLF, XSS	Using one data source
	Landauer et al. (2019)	Netcat Script	Webshell Attack	Limited validity and reliability
	Skopik et al. (2022)	AECID approach	Outliers, Change in Trends	Fluctuating data usage and infrastructure changes
Cyberattack Datasets	Yagemann et al. (2021)	MARSARA	Anomalies	High runtime overhead for PTs
			Spoofing, Delay Attacks	
	Moustafa et al. (2020)	KNN, LOF, Clustering, Correlation	Scanning, Ransomware, DoS, DDoS, Injection, MITM, XSS	No attack detection experiments
	Sharafaldin et al. (2018)	Kali and Windows 8.1 PCs	Brute Force, DoS, DDoS	Using a limited number of PCs
	Mahfouz et al. (2020)	MLP, NB, DT, LR	Brute Force, DDoS, Infiltration	Using one dataset
	Ring et al. (2017)	Nmap and Python Scripts	Port Scans, Brute Force, DoS	Limited to a fixed number of attacks.
	Creech and Hu (2013)	Metasploit, TikiWiki tool	Brute force, password, webshell	Features lack data richness for reliable classification.
	Tovarňák et al. (2020)	Kali Linux VMs	Rreconnaissance, DoS, Web, Phishing, Ransomware, Data leakage	Lake of attack labels
	Landauer et al. (2020)	Port scanning,smtp-user-enum, hydra tool	Port scan, brute force, and webshell	Services dependency and auto labeling issue

more data sources in the data analysis procedure. The event logs are collected from Windows and Linux operating systems to provide a variety of events with nine types of attacks. The event logs are classified based on their messages, which is a missing feature in the body of research. Furthermore, by labeling each event based on its type, this work determines the attack type from textual messages, which aids in precisely determining the attack type. In this study, we explore not only the classification procedure but also how each message contributes to the determination of the attack type. Our approach involves the utilization of explainable AI techniques to enhance transparency and comprehension in the decision-making process of identifying the system's vulnerability to a specific attack type. Finally, the presented dataset is employed to evaluate the efficiency of diverse traditional and deep learning algorithms in the context of detecting cyberattacks.

3. Dataset collection and analysis

3.1. Dataset collection

This study aims to bridge the gap from previous research by following specific patterns to extract significant insights from the available datasets. Despite having few alternatives, we performed an extensive web search and ultimately identified three pertinent event log collections containing text messages that align with the objectives of this study. These datasets are (1) traffic and log data captured during a cyber-defense exercise dataset (TLDCDE) (Tovarňák et al., 2020), (2) austrian institute of technology (AIT) log data (Landauer et al., 2020a), which provides synthetic log events appropriate for analyzing intrusion detection systems, and (3) mid atlantic collegiate cyber defense Competition (MACCDC) dataset (MACCDC, 2012), collected during a competitive test between university and college students. Additionally, a zero-shot dataset of event logs has also been considered in this study, as described next. Cyber-defense exercise and MACCDC

2012 datasets contain event log messages that are recorded during cyberattack testbeds without determining the specific attack for each event message. The ultimate objective is to detect incoming cyberattacks using the acquired knowledge from known attacks to predict and identify new attacks on the network.

3.1.1. TLDCDE dataset

The TLDCDE dataset has a wide range of event logs captured in an exercise network deployed in the KYPO cyber range platform. The captured data was standardized into three unique event categories: archive of IP flow information export (IPFIX) traffic flows; archive of Linux Syslog entries with their text-based log messages, and archive of Windows Event Log entries containing the payload of actual events in raw XML. Each data type has a different number of samples. IPFIX traffic flow data was gathered from various protocols; Linux data was gathered from various domains and applications; and Windows data was gathered from audit events. The exercise is held between two teams: the red team and the blue team. A variety of users and twenty learners were divided into six blue teams to protect an emulated critical information infrastructure from attackers in the red team. To simulate an internal environment, private IP ranges are used to address the blue team networks. Five subcategories represent the blue team network: the demilitarized zone (DMZ), servers (SRV), operations (OPS), users (USR), and the virtual private network (VPN). The exercise aims to expose learners to cybersecurity incidents in a safe, isolated environment. The global gateway is the hub connecting all of the exercise's subnetworks. The exercise scenario uses the global network to imitate the Internet. The service providers and the attackers are the two groups of devices that make up the global network. The service providers enable the blue teams and other genuine virtual participants in the exercise to access legitimate DNS, e-mail, and the web. This network also includes the hub for the exercise's routing and address translation.

3.1.2. AIT log dataset

Another dataset that is used in this work is the AIT dataset. Landauer et al. (2020a) created a new methodology using model-driven engineering (MDE) techniques to collect a new log dataset. A simulation of a typical system infrastructure found in many enterprises was constructed. After analyzing usage statistics of some good technologies, an Apache web server that hosts a mail platform and a content management system (CMS) that is viewed and used by a random number of people was used. Open-source platforms with vulnerabilities, such as Horde groupware webmail and a web shop provided by OkayCMS, were used.

This testbed aims to create log data for assessing attack detection systems. Executing malicious activities on an idle machine, on the other hand, makes identification very easy because practically all created logs are likely to be relevant to the intrusion. Using the prominent web automation tool selenium, a state machine that covers all necessary functions of the Horde webmail and OkayCMS systems was constructed. The logs were gathered from four different testbeds. Each testbed is made up of one web server with a database and a different set of linked host machines, each used by one or more users.

The log data gathered from the web servers contains Apache access, error logs, syscall logs, authentication logs, daemon logs, mail logs, syslog, and user logs. In this testbed, two types of attacks were scheduled: multistep intrusion and webshell injection. The multistep intrusion has multiple tools generally used to exploit vulnerabilities to gain root access to a mail server. Scans for open ports and vulnerabilities are performed as the first two steps. The attacker then employs the SMTP-user-enum tool to discover Horde webmail accounts based on a list of popular names, followed by the Hydra tool to brute-force log into one of the accounts based on a list of standard passwords. The attack continues with a Horde Webmail exploit that enables the upload of a webshell and remote command execution.

3.1.3. MACCDC 2012 dataset

MACCDC (2012) is a one-of-a-kind opportunity for university and college students to use their cybersecurity knowledge and skills in a competitive test. Its distinctiveness lies in its focus on the operational aspects of safeguarding an already established network infrastructure. The logs were collected from the SecRepo2 website. The MACCDC completion has two participating teams and two rounds. Each team alternates between attacking and defending the competition's network. In the first round, team A assaults the network while team B defends it. In the second round, team B assaults the network while team A defends it. Both sides employ their campaigns to attack the network, evading the defensive team's preemptive and preventive strategies. Throughout the contest, the teams must ensure that the systems provide the required services when attacked by a volunteer Red Team. Furthermore, the teams must complete periodic "injects" that imitate business operations that IT professionals must deal with in the real world. The competition's network had 4757 hosts, with 4739 experiencing malicious traffic. This generated 2,677,375 snort alarms across 25 classes (Peshave et al., 2022). The malicious network event trajectories were used to construct sub-sequences of length 10, resulting in 658,343 unique subsequences, out of a total of 51,606 unique subsequences.

3.1.4. Zero-shot dataset

In contrast to the previous datasets, this data was specifically gathered from a local server solely for this study. The data collected on Tuesday, October 25, 2022, from 8:00 a.m. to 3:00 p.m. includes events generated using various systems, including service control manager, Microsoft-Windows-Kernel-General, Microsoft-Windows-TerminalServices-Printers, and Microsoft Windows WAS. It contains Windows event logs representing daily activities, information on the institution's status, performance, and security, as well as information on the apps and services that run on that server. This server gives local hosts quick, dependable access to critical resources and information,

greater control over data management and security, and the ability to tailor services to meet the specific needs of strategic imperatives. This dataset did not contain any suspicious events. The vast majority of the event logs concerned Windows services, boot events, crash events, and processing events. This dataset is exclusively used for event-type detection experiments due to its non-suspicious nature. Table 2 provides an overview of the traffic and log data collected from the four datasets.

3.2. The procedure of data labeling

The labeling process is determined by either attack types or event types. The AIT dataset is already labeled, but the MACCDC dataset and traffic/log dataset need labeling. The dataset obtained from a public local server was labeled solely based on event types due to its lack of suspicious events. A two-stage data labeling process has been employed, utilizing automated labeling based on message content and specific time intervals for each dataset. In the initial labeling phase, all log events generated during the identified attack time intervals has been classified as either suspicious or non suspicious, in accordance with the temporal information provided by the datasets. While this labeling method is straightforward due to the ability to derive anomalous time windows from attack scenario descriptions, it has the drawback of also categorizing non suspicious events occurring during attack periods as suspicious. The researchers in AIT employed a timely detection mechanism for event labeling as a preliminary step, ensuring that this data was already labeled and eliminating the need for additional labeling. In contrast, the TLDCDE dataset contains time intervals without corresponding labels. To address this, we applied similar mechanism to (Landauer et al., 2020a) as initial step to label this data using a script. Employing automated labeling proves advantageous due to its speed and reduced effort requirements.

However, the trade-off for this approach is a notable increase in misclassification, as the code script may struggle to label unseen data. Samples that are not part of the available dataset may not be labeled correctly. Such untrained or unexpected scenarios can be handled by human professionals. Consequently, resorting to manual labeling emerges as a viable solution, even a time-consuming and labor-intensive one, particularly when handling extensive datasets to ensure accurate classification of the data. Landauer et al. (2020a) had difficulty distinguishing between interleaved malicious and normal processes due to the reliance on the code script in their labeling procedure, which resulted in misclassification.

On the other hand, Ramish et al. (2022) and Chung et al. (2023), labeled their data manually to avoid the misclassification that will result from automatic classification. Therefore, we performed manual corrections as a second phase to improve the reliability of the labeled events. The subsequent subsections explain the procedure of manual correction applied to each attack and event type.

3.2.1. Labeling based on attack types

In this work, a wide range of Windows and Linux event log files were obtained to classify them based on the attack types. The log messages were analyzed to discover the threat scenarios followed by the attackers. However, all the event logs could be suspicious, as they were captured in a controlled environment under cyber defense attack conditions. All data was collected when the attacks were actively performed, while users were simultaneously interacting with the system. So the events in this study were classified into suspicious and non-suspicious depending on the log message's semantic meaning and the recommendations provided in monitoring active directory for signs of compromise that were documented by Microsoft.

There is a list of events that should be monitored in any environment to ensure security and detect potential threats. By leveraging established classification methods and industry best practices, the event logs were categorized into suspicious and non-suspicious events. The

Table 2

A description of the traffic and log data collected from the four datasets.

Dataset	Data source	Samples	Info source
TLDCADE	IPFIX traffic flows	469,113	DNS, HTTP, DHCP, SSH, and POP3 protocols.
	Linux	6,083,409	smbd, cron, sshd, usermod, firewall, and useradd daemons and apps.
	Windows	2,901,154	user login/ logout, credentials validation, and privilege escalation audit events.
AIT	Linux	15,817	Apache2, PHP7, Exim, Horde, OkayCMS, Suricata, ClamAV, and MariaDB.
MACCDC	PCAP files	2,677,375	TCP, ICMP, SMP, UDP, HTTP, DHCP, and routing protocols.
Zero-shot	Windows	70,357	User login/ logout, credentials validation, and privilege escalation audit events.

```

>> connect from unknown[10.0.3.33]

>> input_userauth_request: invalid user chalupova.milena

>> Failed password for invalid user novakova.hana from 4.122.55.114 port 56414 ssh2

>> Mar 4 19:25:47 mail auth: pam_unix(dovecot:auth): authentication failure; logname= uid=0 euid=0 tty=dovecot ruser=daryl rhost=127.0.0.1 user=daryl

>> Mar 4 19:29:29 mail HORDE: [horde] PHP ERROR: Cannot modify header information - headers already sent [pid 8580 on line 0 of "Unknown"]

>> An account was logged off. \r\n\r\nSubject:\r\n\tSecurity ID:\t\tS-1-5-21-1922877671-778123372-2883893072-1155\r\n\tAccount Name:\t\tOPS-DESKTOP25\r\n\tAccount Domain:\t\tCHMEL\r\n\tLogon ID:\t\t0x2DB861\r\n\r\nLogon Type:\t\t3\r\n\r\nThis event is generated when a logon session is destroyed. It may be positively correlated with a logon event using the Logon ID value. Logon IDs are only unique between reboots on the same computer.

```

Fig. 1. Example system logs illustrating static and variable message components.

dataset was filtered by removing all the non-suspicious events and starting a new classification to classify each suspicious event based on the attack type. Various attempts have been made to carry out illegal actions while using explicit credentials resulting in many cyberattacks, including reading administrator credentials, numerous attempts to scan open ports, receiving spam emails, obtaining permissions for risky operations, and many more. The event logs were classified into nine types of attack: network reconnaissance attack (NRA), credential based attack (CBA), data leakage attack (DLA), brute force attack (BFA), phishing attack (PHA), pass the ticket attack (PTT), denial of service attack (DoS), web attack (WA) and privilege escalation attack (PEA).

Network reconnaissance attack. Is the foundational step for most types of attacks. The attacker attempts to gather critical information about the victim, such as identifying the system's weak points, the open ports, IP addresses, network topology, domain names, user names, password complexity requirements, and more. As a result, in Windows data, any attempt to read sensitive information in the system carried out by a third party or by the admin/administrator was classified as a network reconnaissance attack because many events in the system involve other parties reading the administrator's credentials. About Linux, all attempts to probe, scan, or send unusual HTTP requests on a specified host, and all Nmap and Nikto events were incorporated into the Network Reconnaissance Attack.

Privilege escalation attack. Aims to expand the users' access permissions for critical tasks such as troubleshooting, financial reporting, or program installation. There are two types of privilege escalation: vertical and horizontal privilege escalation. Vertical privilege escalation involves attackers using limited user accounts to increase their privileges, potentially by assigning special privileges or adding them to a security group, while still using the same account. On the other hand, horizontal privilege escalation allows attackers to access higher-priority admin credentials, allowing them to move around the network without detection. This type of attack found only in Windows event logs, involves assigning special privileges to new accounts or adding new members to security groups. These events were not isolated or routine administrative actions. Instead, they occurred frequently alongside other attack patterns, such as denial-of-service or system exploitation attempts. This behavior suggests a deliberate effort to exploit vulnerabilities and elevate access rights, which is a hallmark of privilege escalation attacks.

Brute force attack. Involves an attacker systematically trying all possible passwords and passphrases until the correct combination is discovered. In Windows data, failed login attempts were classified as brute

force attacks. In Linux, attempts with different credentials, ignored by the PAM service, and attempts exceeding the maximum allowed number by an invalid user were classified as brute force attacks. All authentication failures, attempts taken by an invalid user, and aborted login attempts were also classified as brute force attacks.

Data leakage attack. Is the disclosure of sensitive data to an untrustworthy environment. Data leakage occurs due to cyberattacks, inside job responsibilities, or unintentional data loss or exposure in an organization. As a result, in Windows data, any attempt to read sensitive information in the system carried out by a third party or the admin/administrator was classified as a data leakage attack because many events in the system involve other parties reading the administrator's credentials. Due to the reading of credentials by users who were not the owners, the log-on attempts using explicit credentials were classified as data leakage attacks. In Linux, all the attempts to load the host key, all the attempts to check passwords by unknown users, all the attempts to get a rivest shamir adleman (RSA) certificate, all the TLS library problems that describe illegal usage of data files or certificates, and all the attempts to get AFS tokens were classified as a data leakage attacks.

Pass the ticket attack. Is a cyberattack where an attacker steals and uses Kerberos tickets, specifically the ticket-granting ticket (TGT) or Service Tickets, to gain unauthorized access to a network. Kerberos is an authentication protocol used in enterprise environments to manage secure access to resources. The attacker compromises a system through phishing or exploiting vulnerabilities, extracts Kerberos tickets from the compromised system, and impersonates users to gain access to other systems or services. An attacker can use a pass-the-ticket attack to get a Kerberos ticket-granting ticket (TGT) from LSASS memory on one computer and use it on another to request Kerberos service tickets (TGS) to access the network resources. So, all the Kerberos service ticket requests were classified as pass-the-ticket attacks. This type of attack was found only in the Windows event logs.

Denial of service attack. Is a type of cyberattack in which the attacker attempts to make a machine or network resource inaccessible to the end users by causing service crashes for the connected device. This is done by overwhelming the system with millions of requests to cause system breakdown or application and service crashes. The goal of the attacks was to cause system breakdowns or crashes in applications and services. In Windows, all application or service crashes were classified as denial of service (DoS) attacks, as they disrupted critical functionality and aligned with the intent of denying access to resources. While crashes can sometimes result from software bugs,

the consistent targeting of key services indicated a deliberate effort to compromise system availability. In Linux, several indicators were also classified as DoS attacks, including any attempts to terminate processes using SIGKILL, dropped client connections, unresponsive authentication processes, the inability to reach specific hosts, packet loss, and lost connections after establishing a new connection from an unknown host. These actions undermined the system's ability to maintain service continuity, reflecting the core characteristics of a DoS attack aimed at overwhelming and destabilizing the environment.

Credential based attack. Involves attackers stealing user credentials to gain access to a network, bypass security measures, and steal sensitive information, with the initial step being to steal credentials. In Windows data, all successful or failed login or logout attempts from suspicious IP addresses in the used dataset have a network logon type. As a result, these suspicious events were carried out on the network side. On this side, the attackers have several options for stealing credentials. They can steal them by sending a fake message to trick the user into giving the attacker access to sensitive information. This fake message may contain a forward link to a malicious website, a reset password link, or others. There are a variety of paths the attacker can take, such as phishing, pass-the-hash, data leakage, brute force, and more. Concerning Linux, all the connections and disconnections, all opened and closed sessions, and all the login attempts with the PLAIN method from suspicious IP addresses were classified as credential-based attacks.

Phishing attack. Aims to exploit user vulnerabilities by constantly seeking ways to entice consumers to visit a phishing website. Spoofed emails are often professionally designed to target users and deceive them. Therefore, all the attempts that were made by spammers to hit the mail server with bogus domains were classified as phishing attacks. This type of attack is found only in the Linux event logs.

Web attack. Is unauthorized third-party access to systems or networks, aiming to destroy or steal sensitive data from information systems, frameworks, computer networks, or personal devices. All web-shell attack events were added to the web attack. Attackers can manipulate GET parameters to inject SQL queries into databases, allowing unauthorized data access or modification. They can also use POST data to execute arbitrary commands if the application does not properly sanitize inputs. therefore, all the successful or failed attempts to connect to the server, any disconnection from unknown, any suspicious GET or POST requests, all not permitted operations, all the external client requests from the server with a reserved local IP address to use it as a legal client for 24 h, and all the requests that imply opening new sessions for external clients and providing them with configured information were classified as a web attack. This type of attack was found only in the Linux event logs. All the SQL injections and all the attempts to establish an encrypted connection between a web server and a web browser using the secure sockets layer (SSL) security protocol were classified as web attacks.

Table 3 shows the statistics on attack types within the data suggest that the dataset is predominantly characterized by credential-based attacks, comprising a total of 7401 messages. Conversely, the phishing attack represents the smallest number of messages in the dataset. This can be attributed to the fact that the overwhelming majority of events in the dataset revolved around the utilization of credentials, such as log-in and log-out events.

3.2.2. Labeling based on event types

In this paper, the event logs are obtained from different sources such as Windows, Linux, and network traffic. Each log source has a distinct format suited to its environment. Windows logs are structured with components like Event ID, timestamp, source (e.g., Security, System), user, and a detailed message, typically stored in the EVTX format, which requires tools such as Event Viewer to read. Linux logs are stored as plain text files and follow a simpler format that includes the timestamp, host, process, log level (e.g., info, error), and a message

Table 3
Statistics of attack types dataset.

Attack type	Training	Validation	Testing	Total
Credential-Based Attack	5,599	623	1,179	7,401
Brute Force Attack	2,637	293	1,000	3,930
Privilege Escalation	1,993	222	500	2,715
Network Reconnaissance	1,795	200	500	2,495
Pass-the-Ticket	320	36	150	506
Web Attack	238	26	110	374
Data Leakage	202	22	100	324
Denial of Service	118	13	42	173
Phishing	4	0	4	8
Total	12,906	1,435	3,585	17,926

describing the event. On the other hand, network traffic logs, captured in PCAP files, record network data such as timestamps, source and destination IPs, ports, protocols (TCP, UDP), and packet content. These logs focus on raw network traffic and are analyzed with tools such as Wireshark. Table 4 summarizes the formats for Windows, Linux, and network traffic. Fig. 1 demonstrates example log messages from the collected dataset, showcasing both static and variable components.

All the collected system logs whether suspicious or non-suspicious were manually labeled into eight distinct event types based on the semantic content of the log messages, as they did not come with predefined event classification categories, as detailed in Table 5. For example, events involving account logins, user account creation or deletion, successful or failed logins, or credential authentication were classified under Account Usage. System start-up, shutdown, or update times were categorized as boot events, while the installation of new applications or services fell under software and service installation. Crash events included logs detailing application crashes, errors, or hangs, and actions involving clearing history or files, such as through hypertext preprocessor (PHP), were labeled as clearing events. Any use of Internet message access protocol (IMAP) was classified as a Mailing Event, and logs related to the initiation and completion of processes were grouped under Processing events. This manual labeling ensured consistency across the dataset, addressing the variability in log formats and content. Based on Table 5 that describes the data statistics for the various types of events, it can be observed that the account usage events dominate the dataset, with a total of 14,173 messages. In contrast, the processing events have the fewest messages in the dataset.

Fig. 2 illustrates the event messages (a) and the attack messages (b). The x-axis indicates the message number in the dataset, while the y-axis indicates the number of words in each message. As shown in this figure, the number of messages in the attack types data reaches more than 17,500. The length of messages varies with the number of words. Some of the event messages have more than 5500 words, while others contain no more than five words. The number of the events' messages is enormous; it has surpassed 17,900.

4. Methodology

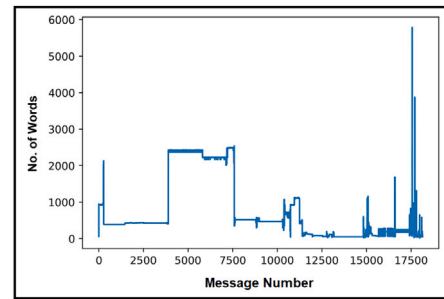
This section discusses the methodology used to analyze and evaluate the collected dataset. Fig. 3 shows the general proposed framework consists of five stages: data collection and preparation which is the initial step that involves gathering data from various sources, preprocessing, cleaning, and transforming it into a suitable analysis format. Feature extraction aims to identify key features that improve model performance and make precise predictions. Architecture construction involves designing and implementing a model architecture, machine learning, or deep learning. After training the model, it is crucial to assess its performance using appropriate metrics to assess its accuracy, reliability, and generalization capabilities on unseen data. Finally, semantic feature analysis involves analyzing the model's predictions and features to comprehend its semantics, interpret results, and provide insights for decision-making using explainable AI tools.

Table 4
A Summary of Log Formats for Windows, Linux, and Network Traffic.

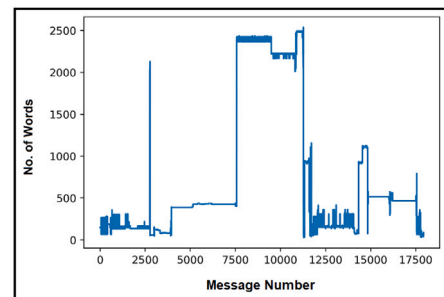
Log type	Components	Format	Example
Windows	- Event ID - Timestamp - Source - User - Event Type - Message	EVTX	An account failed to log on. Subject: Security ID: S-1-0-0 Account Name: - Account Domain: - Logon ID:0 × 0 Logon Type:3 Account ForWhich Logon Failed: Security ID:S-1-0-0 AccountName: User Account Domain: WORKSTATION Failure Information: Failure Reason: The specified account's password has expired. Status: 0xC0000224
Linux	- Timestamp - Host - Process/Service - Log Level - Message	Plain text	Time server_name APPLICATION: model event Authentication failed. [pid 12345 on line 678 of "/path/to/file.php"]
Network Traffic	- Timestamp - Source IP/Port - Destination IP/Port - Protocol - Packet Length - Packet Data	PCAP	Packet No: 1633247 Time: 1706.530000 Source IP: 192.168.X.X Destination IP: 192.168.Y.Y Protocol: TCP Length: 70 Source Port: 50916 → Destination Port: 80 [ACK] Sequence No: 306 Acknowledgment: 464 Window Size: 15680 Length: 0 Timestamp Val: 5734915 Timestamp Echo:1882902

Table 5
Statistics of event types dataset.

Event type	Training	Validation	Testing	Total
Account Usage	10,636	1,182	2,355	14,173
Mailing Events	567	63	310	940
Processing Events	582	65	200	847
Crash Events	624	70	400	1,094
Windows Services	251	28	150	429
Software&Service Installation	146	16	100	262
Boot Events	76	8	40	124
Clearing Events	39	4	30	73
Total	12,921	1,436	3,585	17,942



(a)



(b)

Fig. 2. Event messages' words counted by (a) Event and (b) Attack Messages.

4.1. Data preprocessing

The event logs have been collected from four different sources. The AIT dataset already had labels. The other datasets were unlabeled, so a new framework has been created to include a unique phase for labeling them depending on the attack and event types. The dataset obtained from the local server has been labeled solely based on event types due to its lack of suspicious events. However, after gathering data from several sources, a new dataset that shows a great deal of variety has been constructed and used to evaluate this framework. Fig. 3 shows the preprocessing phase consists of four substeps, starting with data transformation. In this step, the data has several forms. It is converted from JSON to TXT, then to the XML format, and finally to CSV format. The second step involves removing duplicates by eliminating identical events. This step is crucial, given that a substantial portion of the ground truth data was identified as duplicated. In the third step, the data is categorized based on event types, with events being reclassified into suspicious and non-suspicious categories. Finally, the dataset is filtered by removing all the non-suspicious events to start a new type of classification based on the attack types.

4.2. Feature extraction

The data is primarily composed of event messages and their labels; therefore, no standard technique is utilized during the feature selection phase. However, all of the words contained in the text of each

event message were included because they were a crucial aspect of predicting the type of attack or type of event. In the feature extraction phase, CountVectorizer (Manjunatha and Kempanna, 2022) was used to convert a given text into a vector based on the number of times that each word appears across the full text. There were several text messages with variable lengths that needed to turn each word in them into a vector to make them more useful in the text analysis process. On the other hand, the Bag-of-Words technique was employed for deep learning feature extraction. This approach extracts numerical features

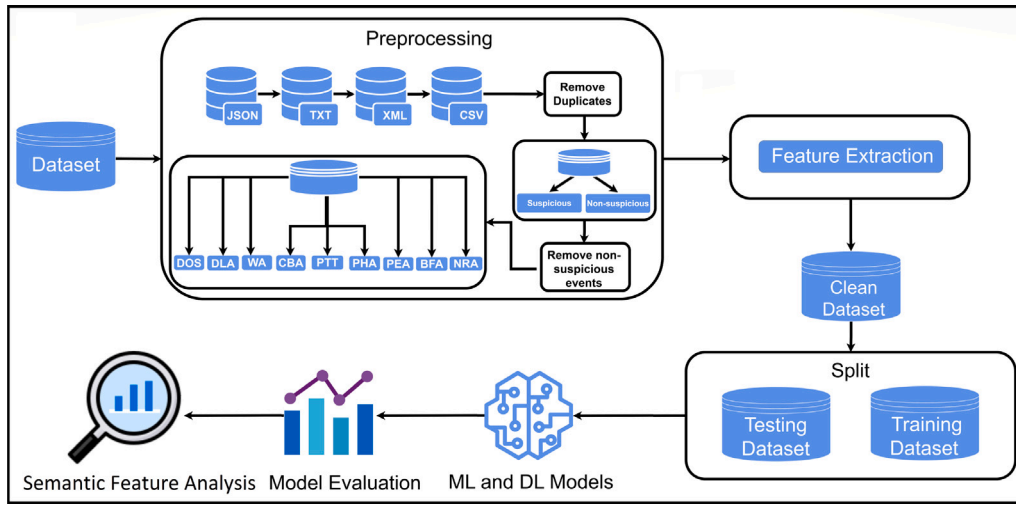


Fig. 3. A depiction of the generic classification framework.

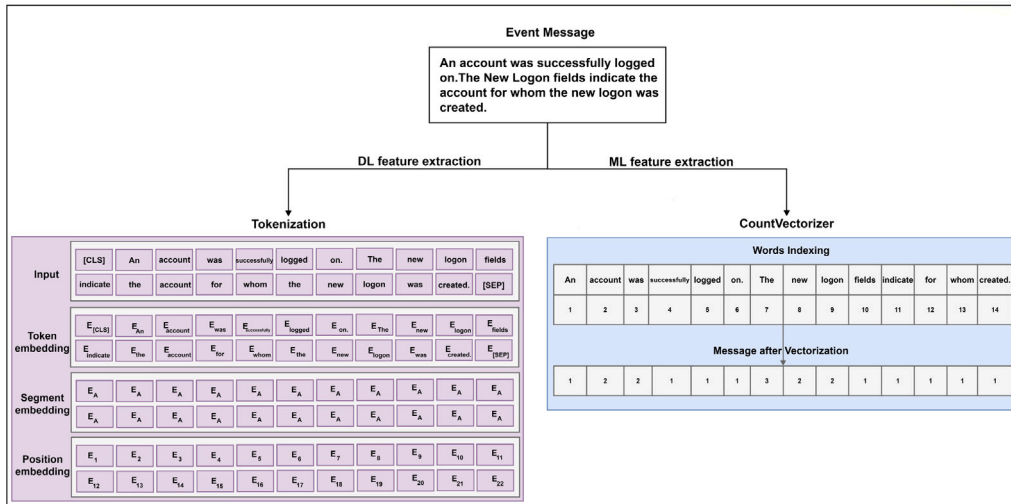


Fig. 4. A block diagram of semantic feature extraction from log messages.

from text material in various ways, including tokenizing, counting, and normalizing. This work employed the tokenizing method, which entails tokenizing strings, assigning a numeric ID to each potential token, and then generating token separators out of white spaces and punctuation. Fig. 4 explains how the countVectorizer and tokenization techniques extract the words from the event message.

4.3. Data split

The data splitting step involved implementing numerous experiments to pick the best data splitting for each used model and get the best results. The data was split into training, validation, and testing datasets. A training dataset of 70%, a validation dataset of 10%, and a testing dataset of 20% were used. Also, a zero random state with stratification was applied to give all the datasets the same proportions of class labels.

4.4. ML and DL modeling

The use of various models in cybersecurity and event log attack detection relies on their unique capabilities to process and analyze

complex, high-dimensional datasets. Traditional machine learning algorithms provide robust classification and predictive capabilities by effectively identifying patterns within event features. These features often represent distinct aspects of system behavior, such as user login attempts, access to sensitive files, and changes in system configurations, each carrying semantic meaning related to user activity and potential security threats. Also, deep learning models excel in capturing intricate spatial and temporal dependencies in the data. These models are highly effective in identifying and mitigating potential cyber threats due to their resilience against overfitting, scalability for large datasets, and enhanced ability to uncover subtle relationships. Integrating machine learning and deep learning techniques enhances the accuracy of detection mechanisms, improving cybersecurity strategies for digital infrastructure protection, and promoting a comprehensive understanding of the semantic meaning behind each feature.

This work used traditional machine learning and deep learning algorithms. For the traditional machine learning for performance evaluation. Seven standard algorithms were applied: multi-layer perceptron (MLP), logistic regression (LR), random forest (RF), support vector classifier (SVC), extreme gradient boosting (XGBoost), stochastic gradient descent (SGD), and naive bayes (NB). For deep learning models, we evaluated the BERT-based model, convolutional neural network (CNN),

bidirectional recurrent neural network (Bi-RNN), long short-term memory (LSTM), bidirectional long short-term memory (Bi-LSTM), bidirectional gated recurrent unit (Bi-GRU), and a hybrid model made up of a CNN layer and a GRU layer.

4.5. Evaluation metrics

Four distinct metrics were used to evaluate the performance of the machine learning and deep learning models, which are: accuracy, precision, recall, and F1-score. Accuracy denotes the ratio of correct predictions to total samples, precision indicates the ratio of correctly classified positive predictions, recall refers to the ratio of the positive samples that were correctly classified to the total number of positive samples, and F1-score analyzes the classification accuracy by considering both the precision and recall metrics. The recall measures the model's ability to identify relevant events for a specific attack type. It is the fraction of true positive events out of all actual positive events. The objective is to maximize true positives while minimizing false negatives when classifying events based on their actual attack type. High recall is particularly important because false negatives (events that belong to an attack type but are misclassified as not belonging to that type) are costly and potentially harmful. For instance, if a suspicious event is misclassified as non-suspicious, it may go undetected by administrators, allowing attackers to continue their activity, potentially causing greater damage to the system. On the other hand, Precision is the accuracy of positive predictions, focusing on the proportion of true positive events out of all classified as positive. High precision is crucial to minimize false positives, where non-suspicious events are incorrectly flagged as suspicious, which can overwhelm administrators and reduce system efficiency. This study emphasizes both recall and precision to ensure that the model not only captures all relevant events but also minimizes unnecessary false alarms. A balance between high recall and precision ensures the model can effectively detect minority classes (i.e., rare attack types) without overwhelming administrators with false positives, which is critical in our scenario where both false negatives and false positives carry significant consequences.

4.6. Semantic feature analysis

Explainable AI techniques shed light on the decision-making process of black-box classifiers and help analysts understand and confidently accept expected outputs (Kasturi, 2024). This work uses explainable techniques to understand the decision-making process of machine/deep learning models that got the best results. The decision made by a DL or ML model has important repercussions in determining the type of attack that the system is vulnerable to, as well as the approach that administrators will take to deal with such an attack (Alzu'bi et al., 2024). We used LIME (Ribeiro et al., 2016) and Captum (Kokhlikyan et al., 2019) to interpret each word in the event message, to explain its impact on the final decision, and to discover biases and incorrect decisions in the predicted data. A thorough study has revealed that these two methods are highly effective in explaining features and can justify any type of prediction made by a supervised learning model (Gohel et al., 2021).

5. Results and discussion

5.1. Experiments configuration

The experiments were conducted on a professional cloud processing platform with a powerful GPU. The proposed event log classifier was implemented using Python and machine learning packages such as TensorFlow and Keras. The client system was equipped with an Nvidia T4 Tensor Core GPU, Intel Core i7 processor, and 12 GB RAM. Each model was trained for a different number of epochs with a learning rate of $2e-5$ regarding BERT. Table 6 summarizes the hyper-parameters used for each model.

5.2. Experimental results

This work conducted three types of experiments: the first was based on the attack types dataset, the second was based on the event types dataset, and the last experiment was based on a zero-shot dataset. A combination of traditional machine learning and deep learning algorithms was employed in all experiments. In each experiment, the models' performance was evaluated using four metrics: accuracy, precision, recall, and f1-score. The following subsections present the models' results based on the dataset types.

5.2.1. Performance based on attack type

With the best Hyperparameters for each used model, the CNN, Bi-LSTM, Bi-GRU, and a hybrid model made up of a CNN layer and a GRU layer were utilized under the same experimental configurations. These models were trained throughout a 10-epoch session with an 8-batch size. The BERT-Based model were trained throughout a 10-epoch session with a 32-batch size. On the other hand, the LSTM and Bi-RNN models were trained over a 7-epoch session with a batch size of 32. Tables 7 and 8 summarize the performance of machine learning and deep learning algorithms in terms of accuracy, weighted precision, weighted recall, weighted f1-score, macro precision, macro recall, and macro f1-score. As demonstrated in Table 8, the XGBoost model obtained the best ML results, with a recall of 98.01, and a precision of 97.94. The results of the ML models were less efficient than the results of the DL models. The BERT classifier achieved the best results, with a recall of 98.77 and a precision of 98.98. The accuracy metric has a sober limitation since it provides values that are less distinct and discriminable. As a result, its capabilities to select and determine the best detection model are hampered. The accuracy is less informative and biased toward the minority class. Therefore, the attention is highlighted on recall and precision results, as they are considered the best discriminators in this context. Recall ensures that the model identifies as many attacks as possible (minimizing missed detections), while precision focuses on reducing false positives (ensuring non-attacks are not mistakenly flagged).

5.2.2. Performance based on event types

With the best hyper parameters for each used model, the CNN, Bi-LSTM, Bi-GRU, and a hybrid model made up of a CNN layer and a GRU layer were utilized under the same experimental configurations. These models were trained throughout a 10-epoch session with an 8-batch size. The BERT-Based model was trained throughout a 10-epoch session with a 32-batch size. On the other hand, the LSTM and Bi-RNN models were trained over a 7-epoch session with a batch size of 32. As shown in Table 9, the SGD model achieved the best ML results, with a recall of 99.05 and a precision of 99.02. The performance of the ML model was less effective compared to the results achieved by the DL model. With a recall of 99.27 and a precision of 99.29, the BERT classifier achieved the best results.

5.3. Discussion

5.3.1. Model optimization for attack detection

This study addresses the effectiveness of machine learning and deep learning algorithms in identifying attacks and events. Transformer-based deep learning models, such as BERT, are especially successful in combating complex and nuanced cybersecurity attacks because they can capture greater contextual connections in data, which is critical for detecting subtle or changing attack trends. Traditional machine learning models including XGBoost and Random Forest are effective in detecting attacks, with accuracy close to deep learning models. These methods allow rapid and resource-efficient detection, making them ideal for real-time threat monitoring. Balancing detection rates among attack types is critical, and models excelling in macro measures, such as BERT, are particularly successful at detecting less common

Table 6

The hyper-parameters of the implemented DL models.

Hyper-param.	BERT	CNN	Bi-LSTM	LSTM	Bi-GRU	Hybrid	Bi-RNN
Loss	Entropy	Entropy	Entropy	Entropy	Entropy	Entropy	Entropy
Activation	SM,R	SM,R	SM	SM	SM	SM,R	SM/Tanh
Optimizer	Adam	Adam	Adam	Adam	Adam	Adam	Adam
Epochs	10	10	10	7	10	10	7
Batch Size	32	8	8	32	8	8	32
Embedding	768	100	32	100	100	100	100
Dropout	0.2	0.2	0.4	0.5	0.25,0.5	0.25,0.3,0.5	0.2

SM: Softmax, R: ReLU, Tanh: Hyperbolic Tangent.

Table 7

A summary of the ML and DL results.

Data type	Model	Accuracy	Weighted average (%)		Macro average (%)	
			Precision	F1-score	Precision	F1-score
Attack	XGBoost	98.01	97.94	97.96	96.86	96.27
Type	BERT	98.77	98.98	98.81	96.86	97.56
Event	SGD	99.05	99.02	99.03	97.26	96.82
Type	BERT	99.27	99.29	99.27	97.14	97.51

Table 8

A summary of the experimental results for the attack types.

Type	Model	Weighted average (%)		Weighted macro (%)	
		Precision	Recall	Precision	Recall
ML	MLP	97.66	97.68	95.72	95.10
	LR	97.69	97.68	95.31	95.27
	SVC	95.39	95.67	83.70	78.83
	RF	97.04	97.01	94.14	93.69
	SGD	97.51	97.40	94.81	95.23
	NB	95.94	95.95	96.26	89.29
	XGBoost	97.94	98.01	96.86	95.80
DL	BERT	98.98	98.77	96.86	98.63
	Bi-LSTM	97.92	97.68	96.91	94.84
	LSTM	97.61	97.60	86.45	84.45
	CNN	96.99	96.51	94.87	96.23
	Bi-GRU	97.11	96.93	96.97	96.42
	Hybrid	96.61	96.51	96.42	95.07
	Bi-RNN	96.37	95.98	97.00	82.80

Table 9

A summary of the experimental results for the event types.

Type	Model	Weighted average (%)		Weighted macro (%)	
		Precision	Recall	Precision	Recall
ML	MLP	98.57	98.57	95.19	94.98
	LR	98.64	98.66	97.13	96.03
	SVC	97.49	97.57	96.89	91.61
	RF	98.61	98.63	96.59	95.98
	SGD	99.02	99.05	97.26	96.42
	NB	96.79	96.54	90.90	93.03
	XGBoost	98.31	98.38	96.58	94.50
DL	BERT	99.29	99.27	97.14	98.08
	Bi-LSTM	98.89	98.74	96.31	95.62
	LSTM	99.00	98.68	97.01	95.24
	CNN	99.02	98.77	94.44	95.72
	Bi-GRU	98.22	98.04	96.08	93.44
	Hybrid	97.83	97.65	92.96	90.61
	Bi-RNN	96.92	96.40	88.80	89.48

but vital attacks, which is necessary for reliable defense systems. In cybersecurity, it is important to detect anomalies and hostile activity with minimal delay and high accuracy. Models must balance computing efficiency and detection performance, especially in real-time networks.

Sajid et al. (2024) outlined the difficulty of using deep learning models, especially hybrid models, which are accurate but have high computational costs and data volumes and face challenges in large-scale networks due to trade-offs between vertical and horizontal scalability. These models can create processing delays, limiting their usefulness in high-speed network traffic situations. We addressed the limitations of

Table 10

A summary of training and testing times complexity for the DL models.

Model	Attack type (s)		Event type (s)	
	Training	Inference	Training	Inference
BERT	290	48	290	49
BiLSTM	32	2	176	12
BiRNN	703	20	1523	141
BiGRU	45	3	290	13
LSTM	1089	32	2272	250
CNN	7	0.002	10	1
Hybrid	18	1.5	95	4

previous work by decreasing the model training time and focusing on one critical feature, the semantic context of the log message, instead of relying on numerous numerical features. This approach reduces the computing costs and training durations and enables a better understanding of patterns and detection of attacks. It also allows for faster training and inference while maintaining high accuracy in real-time attack detection.

This technique maximizes computational efficiency and real-time performance, addressing scaling difficulties in previous approaches. As shown in Table 10, BERT effectively detects attack and event types, but its high computational cost makes it less suitable for real-time detection. BiLSTM offers a more efficient alternative with a shorter training time of 5 min (32 s per epoch) for attack-type detection and 29.3 min (176 s per epoch) for event-type detection. Its low time complexity makes it suitable for log attack detection. On the other hand, BiRNN has higher computational costs with training times of 82 min (703 s per epoch) for attack type and 177.6 min (1523 s per epoch) for event type, limiting its applicability in real-time intrusion detection. BiGRU is a strong candidate for attack and event type recognition, with training times of 7.5 and 48.3 min, respectively. It incorporates performance and speed, making it a viable option. LSTM models require longer training time because of inherent gating mechanisms, but CNNs have outstanding speed with less training time. CNNs are often employed for image data, yet they offer the potential for fast anomaly detection in cybersecurity activities. The hybrid model, which combines complexity and performance, reaches a compromise, with training durations of 3 min for attack type and 15.8 min for event type detection. The hybrid technique minimizes training time and solves the problem of high computational costs in (Sajid et al., 2024), without compromising much performance. This paper uses model design optimization to achieve a balance between computational efficiency and real-time performance. The main strategies include decreasing training time, balancing complexity and speed, and emphasizing scalability.

Models including BiLSTM, CNN, and Hybrid surpass others in terms of speed while maintaining accuracy. The CNN model takes just 1.16 min to detect attack types, making it perfect for real-time applications. The hybrid model, which keeps the benefits of complex architectures while reducing computing, has a training period of only 3 min for threat detection, indicating a possible middle ground for real-time cybersecurity applications. Focusing on models that demand fewer resources allows these models to scale more efficiently in dispersed contexts, managing high quantities of network traffic without bottlenecks.

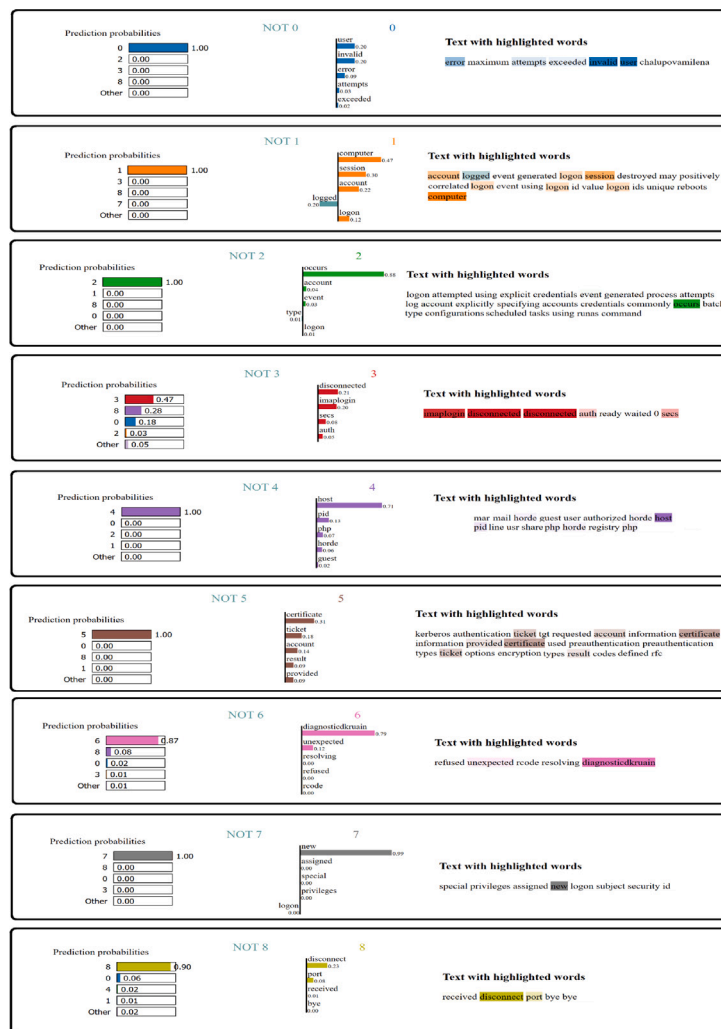


Fig. 5. Message explanations for each attack class.

Table 11 compares the performance of several models in terms of the number of classes, training, and inference time.

Our CNN-based detection model surpasses the other models in terms of training time, taking only 7 s to complete the procedure. In comparison, the LSTM (Sajid et al., 2024), DCNNBiLSTM (Hnamte and Hussain, 2023), and ADS (Muna et al., 2018) models require 73.41, 202, and 119.03 s, respectively. This demonstrates how efficient the CNN model is in learning patterns from data. Inference times differ greatly between models. The DCNNBiLSTM model has the longest inference time (95.04 s), while the ADS model has a little longer inference time (2.25 s) than our CNN model, which takes just 2 ms. This shows that our CNN model not only trains faster but also predicts more accurately, making it more suited for real-time applications. Our CNN model's performance in terms of training and inference time shows that it is more scalable and applicable to large-scale or real-time applications. The short inference time, in particular, makes it perfect for scenarios requiring rapid decision-making, such as online classification or real-time monitoring. Previous research (Sajid et al., 2024; Hnamte and Hussain, 2023; Muna et al., 2018) lacks model interpretability, which is critical for decision-making. Complex models, such as deep learning, can provide complex predictions, making it difficult for users to trust or evaluate the results. This lack of transparency makes it difficult to explain the origins of some predictions. To solve this issue, we included XAI tools in our model pipeline, which give clear, transparent explanations for our models' predictions and insights into the features that influenced each decision. This boosted trust in the model's

outcomes and assured decision-makers could depend on them in critical situations. XAI successfully balances performance and openness, which are essential to preserving accuracy and accountability in real-time cybersecurity systems, assuring the advancement of sophisticated models without compromising interpretability.

5.3.2. Semantic message explanation

This study used LIME tool to interpret the XG-Boost model predictions by creating human-readable explanations. Fig. 5 depicts local explanations for a specific instance of each class label in the data to identify the input data features that were most important in formulating a given prediction. The highlighted words in the text message contributed the most to the prediction process. The greater the color intensity, the stronger the effect. Each word contributes either positively or negatively to the prediction process. If the word appears under the number of the label class, it has a positive effect. If it is listed in the NOT label class, it signifies that it perplexed the model and referred to another class label, which leads to an incorrect prediction. The class labels are encoded and represented numerically, such that 0 indicates a brute force attack, 1 indicates a credential-based attack, 2 indicates a data leakage attack, 3 indicates a denial of service attack, 4 indicates a network reconnaissance attack, 5 indicates a pass-the-ticket attack, 6 indicates a phishing attack, 7 indicates a privilege escalation attack, and 8 indicates a web attack. The labels brute force attack (0), credential-based attack (1), data leakage attack (2), denial of service attack (3), and web attack (8), have a vast majority of the words that

Table 11
A comparison of model performance with prior works.

Ref	Model	No. of Classes	Training (s)	Inference (s)
Sajid et al. (2024)	LSTM	5	73.41	—
Hnamte and Hussain (2023)	DCNNBiLSTM	3	202	95.04
Muna et al. (2018)	ADS	10	119.03	2.25
Ours	CNN	9	7	0.002

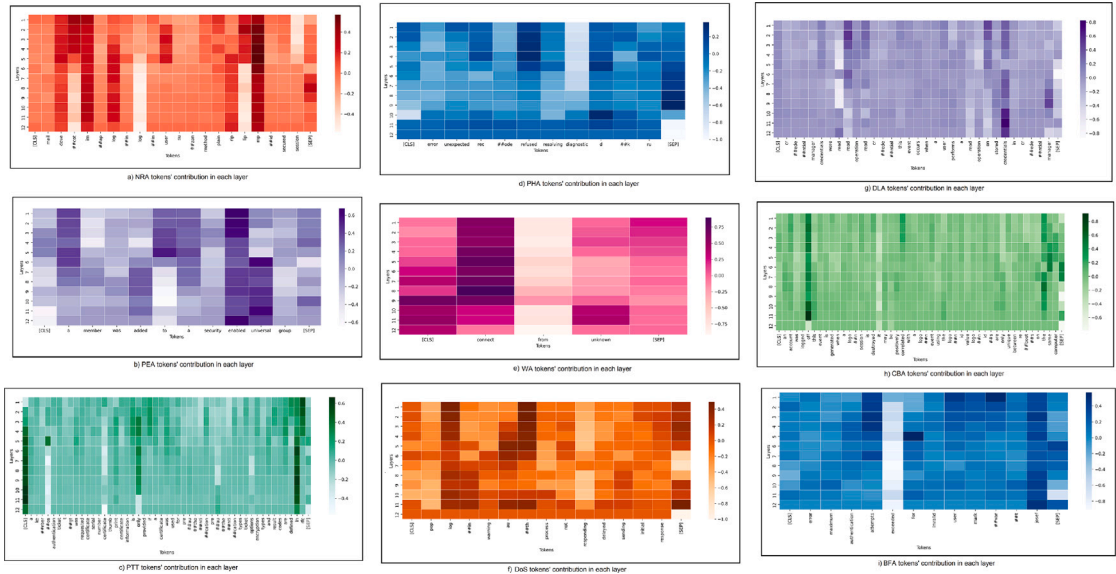


Fig. 6. A representation of the token contribution in BERT layers.

contributed positively to the prediction process. The words “*user*” and “*invalid*” contribute the most to the brute force attack, and this label has a prediction probability of 100%. The most significant word in credential-based attacks was “*computer*”, which has a weight of 0.47. The prediction probability for this label is also 100%. Data leakage attack, network reconnaissance attack, pass-the-ticket attack, and privilege escalation attack have the same prediction probabilities. On the other hand, denial of service attack, phishing attack, and web attack have variant probabilities due to the bias for other labels, exhibiting prediction probabilities of 0.47, 0.87, and 0.90, respectively. Words such as “*type*”, “*logon*”, and “*logged*” have a negative impact on the prediction results in several class labels due to their repetition in event messages that belong to various attacks.

Fig. 6 depicts a heat map of token attributions for the BERT prediction across all layers. It is worth noting that the word “*authentication*” has increasing attribution from layer one to layer nine in the BFA class. This significance is gradually fading in the last three layers. On the other hand, the word “*credentials*” in DLA gets an increasing level of attribution from the first to the last layer.

In contrast to the “*credentials*” token, the “*account*” word in the CBA class continues to decrease. In the first five layers, many tokens in all classes have negative or near-to-zero attribution. It is possible to begin noticing slightly higher attribution in tokens crucial after layer five, such as the PEA class’s “*universal*” word. In the first nine layers, the token “*account*” in the WA class is also granted a high attribution score. Finally, in the last three layers, the correctly predicted token “*error*” for the PHA class has increasingly positive attribution. Since the input text (message) can be handled as a discrete random variable because it has a finite number of possible values, the probability mass function (PMF) is calculated using the attribution values of each feature after extracting the feature importance using a probability distribution on the input features and scaling the attribution values accordingly.

The PMF is used to comprehend how a particular token contributes to the model’s output and to understand its relative importance in each

layer. In the context of BERT, PMF refers to the output layer prediction, where each node indicates the probability of a specific token in the text given the input. In this study, the PMF is calculated utilizing the attribution values of the best word that contributed positively in each class. Fig. 7 shows the PMF for all class labels in the last layer of BERT is high because this is where the model makes its final prediction.

The output layer is a fully connected layer that computes the probability distribution over the entire text using a softmax activation function. The last layer identifies the most likely token (word) given the input and context provided by the rest of the BERT model. High PMF values at the BERT’s final layer imply that the model is confident in its predictions for a certain token in the given text. Attribution entropy is a measure of feature attribution confusion that can be used to detect circumstances where a model’s predictions are less accurate or more difficult to explain. In this work, the attribution entropies are calculated using the Shannon entropy. The Shannon entropy can be used in the context of BERT to quantify the randomness or uncertainty of the model’s predictions for a given token. The amount of uncertainty in a probability distribution is measured by the Shannon entropy. The entropy is calculated from a set of probabilities as the negative sum of the probabilities times their logarithm to base 2.

The probability distribution in the instance of BERT is represented by the prediction made by the last layer, PMF, for a specific token. The PMF’s entropy indicates how confident the model is in its prediction for that token. High entropy indicates uncertainty or less confidence in a model’s prediction, while low entropy indicates certainty and confidence. Fig. 8 depicts the attribution entropies for each class label based on the Shannon entropy measure. The x-axis represents the number of layers, while the y-axis represents the entropy value in each layer.

5.4. Limitations of the study

There are certain limitations to this study, the most prominent of which was encountered during the data processing stage. Upon

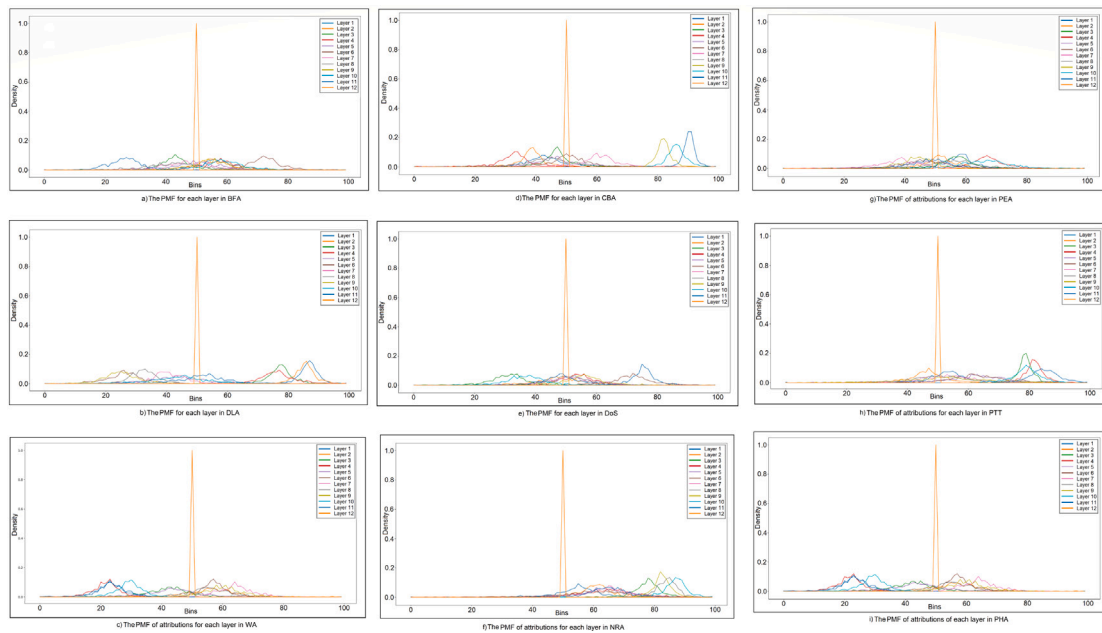


Fig. 7. A representation of the probability mass function (PMF) distributions across BERT layers.

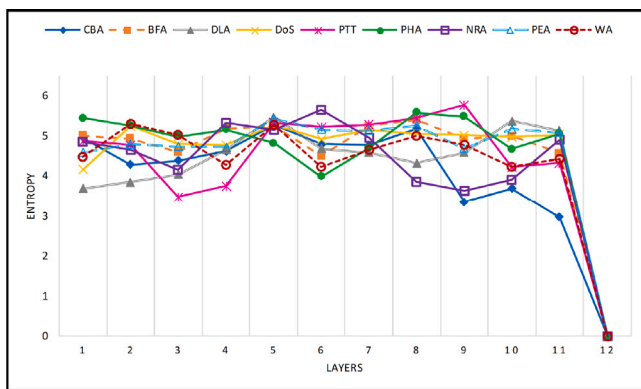


Fig. 8. Attribution entropies for class labels calculated using the Shannon entropy measure.

compiling log data from multiple sources, we found plenty of duplicate messages. The dataset became more noisy due to this redundancy. We cleaned the data by removing duplicate log entries in order to address this problem. This reduced the size of the dataset used for model testing and training. Cyberattacks are constantly evolving, and attackers routinely discover new tactics to evade detection systems. This study's reliance on historical log data, which might not detect all new or developing threats, presents another difficulty. The model's capacity to respond to or identify previously unknown threats is restricted unless it is continuously retrained on up-to-date data. Additionally, since the primary objective of this study is to analyze the semantic context of the log message, which focuses on the static parts, the variable components have not been included in the analysis. A potential solution to address this limitation could involve adopting a pattern similar to the one proposed in (Wurzenberger et al., 2024) which presents an event-based detection mechanism. This method considers all token data types associated with a specific event type during the log message analysis.

6. Conclusion

This paper presents a new event log dataset that is compiled from four separate datasets and categorized according to event and attack

types. By recognizing and monitoring significant events and classifying each event based on the message log, aiding customers in better understanding their events, strengthening security, and minimizing performance degradation would be guaranteed. The suggested framework will assist administrators in preventing incoming threats before critical data is stolen or lost by providing a better understanding of security events. If the topic of interest is not related to event messages, the same framework can still be used by modifying the obtained features and replacing them with other features of events. The results showed that the BERT-Based model outperformed all the conventional and deep learning models evaluated in this work with a recall of 99.27% and a precision of 99.29%. These findings reflect the ideal model's ability to properly identify a high proportion of events of a particular type by detecting as many true positives as possible while avoiding false negatives. Several fascinating potential future work directions including collecting more log samples from intrusion detection systems by developing an attacking simulation session with good system infrastructure. The use of network traffic data and the features of IP flows could be also investigated.

CRedit authorship contribution statement

Ahmad Alzu'bi: Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Omar Darwish:** Writing – review & editing, Writing – original draft, Validation, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **Amjad Albashayreh:** Writing – original draft, Software, Investigation, Formal analysis, Data curation. **Yahya Tashtoush:** Writing – review & editing, Validation, Methodology, Investigation, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Ali, A., Khan, A., Ahmed, M., Jeon, G., 2022. BCALS: Blockchain-based secure log management system for cloud computing. *Trans. Emerg. Telecommun. Technol.* 33 (4), e4272.
- Alzu'bi, A., Albashayreh, A., Abuarqoub, A., Alfawair, M., 2024. Explainable AI-based DDoS attacks classification using deep transfer learning. *Comput. Mater. Continua* 80, 3785–3802. <http://dx.doi.org/10.32604/cmc.2024.052599>.
- Azahari, A., Balzarotti, D., 2024. On the inadequacy of open-source application logs for digital forensics. *Forensic Sci. Int. Digit. Investig.* 49, 301750.
- Ceron, J.M., Scholten, C., Pras, A., Santanna, J., 2020. MikroTik devices landscape, realistic honeypots, and automated attack classification. In: NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–9.
- Chung, M.-H., Wang, L., Li, S., Yang, Y., Giang, C., Jerath, K., Raman, A., Lie, D., Chignell, M., 2023. Implementing active learning in cybersecurity: Detecting anomalies in redacted emails. *arXiv preprint arXiv:2303.00870*.
- Creech, G., Hu, J., 2013. Generation of a new IDS test dataset: Time to retire the KDD collection. In: 2013 IEEE Wireless Communications and Networking Conference. WCNC, IEEE, pp. 4487–4492.
- Das, S., Agarwal, N., Shiva, S., 2021. Ddos explainer using interpretable machine learning. In: 2021 IEEE 12th Annual Information Technology, Electronics and Mobile Communication Conference. IEMCON, IEEE, pp. 0001–0007.
- Das, S., Ashrafuzzaman, M., Sheldon, F.T., Shiva, S., 2020. Network intrusion detection using natural language processing and ensemble machine learning. In: 2020 IEEE Symposium Series on Computational Intelligence. SSCI, IEEE, pp. 829–835.
- Gohel, P., Singh, P., Mohanty, M., 2021. Explainable AI: current status and future directions. *arXiv preprint arXiv:2107.07045*.
- Gunawan, T.S., Lim, M.K., Kartiwi, M., Malik, N.A., Ismail, N., 2018. Penetration testing using kali linux: SQL injection, XSS, wordpres, and WPA2 attacks. *Indones. J. Electr. Eng. Comput. Sci.* 12 (2), 729–737.
- Hnamte, V., Hussain, J., 2023. DCNNBiLSTM: An efficient hybrid deep learning-based intrusion detection system. *Telematics Inform. Rep.* 10, 100053.
- Jaafar, F., Nicolescu, G., Richard, C., 2016. A systematic approach for privilege escalation prevention. In: 2016 IEEE International Conference on Software Quality, Reliability and Security Companion. QRS-C, IEEE, pp. 101–108.
- Kalutharage, C.S., Liu, X., Chrysoulas, C., Pitropakis, N., Papadopoulos, P., 2023. Explainable AI-based DDoS attack identification method for IoT networks. *Computers* 12 (2), 32.
- Kasturi, S., 2024. Predicting Application Security Attack Paths Using Correlation Analysis, Attack Tree, and Multi-layer Perceptron (Ph.D. thesis). Indiana State University.
- Kokhlikyan, N., Miglani, V., Martin, M., Wang, E., Reynolds, J., Melnikov, A., Lunova, N., Reblitz-Richardson, O., 2019. Pytorch captum. GitHub repository.
- Kumar, R., Javeed, D., Aljuhani, A., Jolfaei, A., Kumar, P., Islam, A.N., 2023. Blockchain-based authentication and explainable AI for securing consumer IoT applications. *IEEE Trans. Consum. Electron.*
- Landauer, M., Skopik, F., Frank, M., Hotwagner, W., Wurzenberger, M., Rauber, A., 2022. Maintainable log datasets for evaluation of intrusion detection systems. *IEEE Trans. Dependable Secure Comput.* 20 (4), 3466–3482.
- Landauer, M., Skopik, F., Wurzenberger, M., Hotwagner, W., Rauber, A., 2019. A framework for cyber threat intelligence extraction from raw log data. In: 2019 IEEE International Conference on Big Data. Big Data, IEEE, pp. 3200–3209.
- Landauer, M., Skopik, F., Wurzenberger, M., Hotwagner, W., Rauber, A., 2020a. Have it your way: Generating customized log datasets with a model-driven simulation testbed. *IEEE Trans. Reliab.* 70 (1), 402–415.
- Landauer, M., Skopik, F., Wurzenberger, M., Rauber, A., 2020b. System log clustering approaches for cyber security applications: A survey. *Comput. Secur.* 92, 101739.
- Lin, Y.-D., Wang, Z.-Y., Lin, P.-C., Nguyen, V.-L., Hwang, R.-H., Lai, Y.-C., 2022. Multi-data-source machine learning in intrusion detection: Packet flows, system logs and host statistics. *J. Inf. Secur. Appl.* 68, 103248.
- MACCDC, 2012. Capture files from mid-atlantic CCDC. <https://www.netresec.com/?page=MACCDC>. (Accessed on: 2 March 2023).
- Mahfouz, A., Abuhusseini, A., Venugopal, D., Shiva, S., 2020. Ensemble classifiers for network intrusion detection using a novel network attack dataset. *Future Internet* 12 (11), 180.
- Manjunatha, K., Kempanna, M., 2022. Count vectorizer model based web application vulnerability detection using artificial intelligence approach. *J. Discrete Math. Sci. Cryptogr.* 25 (7), 2039–2048.
- Mei, R., Yan, H.-B., Han, Z.-H., Jiang, J.-C., 2021. CTScopy: hunting cyber threats within enterprise via provenance graph-based analysis. In: 2021 IEEE 21st International Conference on Software Quality, Reliability and Security. QRS, IEEE, pp. 28–39.
- Meng, F., Lou, F., Fu, Y., Tian, Z., 2018. Deep learning based attribute classification insider threat detection for data security. In: 2018 IEEE Third International Conference on Data Science in Cyberspace. DSC, IEEE, pp. 576–581.
- Moustafa, N., Ahmed, M., Ahmed, S., 2020. Data analytics-enabled intrusion detection: Evaluations of ToN_IoT linux datasets. In: 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications. TrustCom, IEEE, pp. 727–735.
- Muna, A.-H., Moustafa, N., Sitnikova, E., 2018. Identification of malicious activities in industrial internet of things based on deep learning models. *J. Inf. Secur. Appl.* 41, 1–11.
- Padiet, P., Islam, R., Khan, M.A., 2024. Users' scenario-base for analysing insider threat detection based on user's downloads activity logs. In: Future of Information and Communication Conference. Springer, pp. 457–472.
- Peshave, A., Ganesan, A., Oates, T., 2022. Predicting network threat events using HMM ensembles. In: International Conference on Advanced Data Mining and Applications. Springer, pp. 229–240.
- Ramish, M., Sinha, A., Desai, J., Raj, A., Rajawat, Y.S., Punia, P., 2022. IT attack detection and classification using users event log feature and behavior analytics through Fourier EEG signal. In: 2022 IEEE 11th International Conference on Communication Systems and Network Technologies. CSNT, IEEE, pp. 577–582.
- Ribeiro, M.T., Singh, S., Guestrin, C., 2016. "why should I trust you?" Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1135–1144.
- Ring, M., Wunderlich, S., Grödl, D., Landes, D., Hotho, A., 2017. Flow-based benchmark data sets for intrusion detection. In: Proceedings of the 16th European Conference on Cyber Warfare and Security. ACPI. pp. 361–369.
- Sajid, M., Malik, K.R., Almogren, A., Malik, T.S., Khan, A.H., Tanveer, J., Rehman, A.U., 2024. Enhancing intrusion detection: a hybrid machine and deep learning approach. *J. Cloud Comput.* 13 (1), 123.
- Satılmış, H., Akleyilek, S., Tok, Z.Y., 2024. A systematic literature review on host-based intrusion detection systems. *IEEE Access* 12, 27237–27266.
- Satpathi, S., Deb, S., Srikant, R., Yan, H., 2019. Learning latent events from network message logs. *IEEE/ACM Trans. Netw.* 27 (4), 1728–1741.
- Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., et al., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1, 108–116.
- Singh, R., Kumar, H., Singla, R.K., Ketti, R.R., 2017. Internet attacks and intrusion detection system: A review of the literature. *Online Inf. Rev.* 41 (2), 171–184.
- Skopik, F., Wurzenberger, M., Höld, G., Landauer, M., Kuhn, W., 2022. Behavior-based anomaly detection in log data of physical access control systems. *IEEE Trans. Dependable Secure Comput.* 20 (4), 3158–3175.
- Sriram, S., Vinayakumar, R., Alazab, M., Soman, K., 2020. Network flow based IoT botnet attack detection using deep learning. In: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops. INFOCOM WKSHPS, IEEE, pp. 189–194.
- Tovarnák, D., Špaček, S., Vykopal, J., 2020. Traffic and log data captured during a cyber defense exercise. *Data Brief* 31, 105784.
- Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S., 2019. Deep learning approach for intelligent intrusion detection system. *IEEE Access* 7, 41525–41550.
- Vu, Q.H., Ruta, D., Cen, L., 2019. Gradient boosting decision trees for cyber security threats detection based on network events logs. In: 2019 IEEE International Conference on Big Data. Big Data, IEEE, pp. 5921–5928.
- Wurzenberger, M., Höld, G., Landauer, M., Skopik, F., 2024. Analysis of statistical properties of variables in log data for advanced anomaly detection in cyber security. *Comput. Secur.* 137, 103631.
- Yagemann, C., Noureddine, M.A., Hassan, W.U., Chung, S., Bates, A., Lee, W., 2021. Validating the integrity of audit logs against execution repartitioning attacks. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. pp. 3337–3351.
- Zhang, P., He, F., Zhang, H., Hu, J., Huang, X., Wang, J., Yin, X., Zhu, H., Li, Y., 2023. Real-time malicious traffic detection with online isolation forest over sd-wan. *IEEE Trans. Inf. Forensics Secur.* 18, 2076–2090.
- Zhang, Y., Wang, L., Liu, Z., Wei, W., 2020. A cyber-insurance scheme for water distribution systems considering malicious cyberattacks. *IEEE Trans. Inf. Forensics Secur.* 16, 1855–1867.

Ahmad Alzu'bi (SMIEEE, MACM) received the M.Sc. degree in computer science from the Jordan University of Science and Technology, Jordan, in 2009, and the Ph.D. degree in computer science from the University of the West of Scotland (UWS), U.K., in 2016. He is currently an Associate Professor with the Department of Computer Science, Jordan University of Science and Technology, Jordan. He has many publications in reputed journals and conferences. His research interests include computer vision, cybersecurity, and artificial intelligence.

Omar Darwish (SMIEEE) received the M.S. degree from the Jordan University of Science and Technology, Jordan, and the Ph.D. degree in computer science from Western Michigan University, USA. He is an Assistant Professor with the Information Security and Applied Computing Department, Game Above College of Engineering and Technology, Eastern Michigan University. He was a Program Coordinator of computer information systems, and the Director of the IoT and Cybersecurity Laboratory, Ferrum College. His research interests include cybersecurity, IoT, machine learning, networks, big data analysis, cloud computing, artificial intelligence, data mining, and information retrieval.

Amjad Albashayreh received her B.S. degree in Software Engineering from Jordan University of Science and Technology (JUST), Jordan in 2020, and her M.S. in

Computer Science from JUST in 2023. She is currently acting as a lecturer with the department of Computer Science, JUST, and she is also pursuing her Ph.D. in Computer Science at The University of Jordan. Her research interests include Cybersecurity, computer vision, deep learning, federated learning, and artificial intelligence.

Yahya Tashtoush received the B.Sc. and M.Sc. degrees in electrical engineering from the Jordan University of Science and Technology, Irbid, Jordan, in 1995 and 1999, respectively, and the Ph.D. degree in computer engineering from the University of Alabama in Huntsville and the University of Alabama at Birmingham, AL, USA, in 2006. He is currently a Professor at the College of Computer and Information Technology, Jordan University of Science and Technology, Irbid, Jordan. His current research interests include software engineering, cybersecurity, artificial intelligence, robotics, and wireless sensor networks.